TH3 — Storage, Register File, Status Register

CS2300 · Group G06
Chaithanya PS (CS25B018)
Yash T(CS25B095)

1. Interface table

Every subcircuit in G__th3_storage.circ and its ports, extracted directly from the submitted file.

Subcircuit	Port	Width	Notes
dff	D / CLK / EN / RST	1	primitive D-latch behaviour on rising CLK; EN gates capture, RST is asynchronous clear
	Q	1	
register16	D / CLK / LOAD / RST	16 / 1 / 1 / 1	16 × dff, common LOAD — matches spec exactly
	Q	16	
regfile8x16	RA, RB, RD	3 each	register index for read port A, read port B, write port
	D	16	write data
	WE, CLK, RST	1 each	
	RA_en, RB_en	1 each	extra output-enables on the two read ports, beyond the base spec — tri-states QA/QB
	EXT	1	purpose not yet documented — see Section 2
	QA, QB	16 each	combinational reads off 8 × register16, gated through tri-state Controlled Buffers
status_reg	Z_IN, N_IN, C_IN, V_IN	1 each	
	STATUS_WE, CLK, RST	1 each	
	Z, N, C, V	1 each	matches spec exactly
ram_word	DIN, ADDR	16 each	ADDR[0] now dropped by a Splitter before reaching the RAM; RAM addrWidth=15 — see Section 4, Fault 2 (resolved)
	MEM_WRITE, MEM_READ, CLK	1 each	
	DOUT	16	

2. Design decisions

2.1 Read-port structure in regfile8x16

The register file is built from eight instances of register16 (not raw dffs), with each register's Q output fed onto both read buses through tri-state Controlled Buffers gated by a 3-to-8 Decoder on RA and RB. This follows the composition requirement from the course conventions and reuses a verified block rather than re-implementing storage.

Two extra ports, RA_en and RB_en, gate the read buses independently of the address decode, so each port can be driven to high impedance rather than always presenting a value. This anticipates the tri-state discipline required of the shared bus from TH4 onward.

2.2 Simultaneous read and write behaviour

Both read ports are combinational: QA/QB are continuously driven from each register's Q output through the tri-state buffers, with no latch on the read path itself. Consequently, when RD = RA (or RB) and WE = 1, the read port reflects the previous value up to the active clock edge and the newly written value immediately afterward — the read is not frozen mid-cycle. This should be stated explicitly in the report and backed by the test in Section 3.4, since the assignment specifically singles out this behaviour for grading.

2.3 Byte ordering

ram_word now drops ADDR[0] with a Splitter before it reaches the RAM primitive (addrWidth reduced from 16 to 15), so ADDR and ADDR+1 correctly resolve to the same 16-bit word — see Section 4, Fault 2 for how this was found and fixed. That settles which addresses share a word; it does not yet say which half of the word each address names, which is the actual byte-ordering (endianness) decision and only becomes observable once a byte-lane mux is added in TH6.

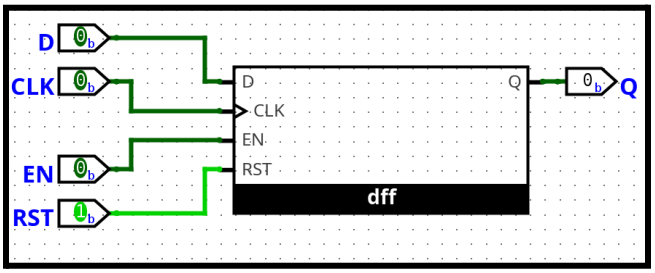
Decision: little-endian. The even address (ADDR[0] = 0) names the low byte, bits [7:0]; the odd address (ADDR[0] = 1) names the high byte, bits [15:8].

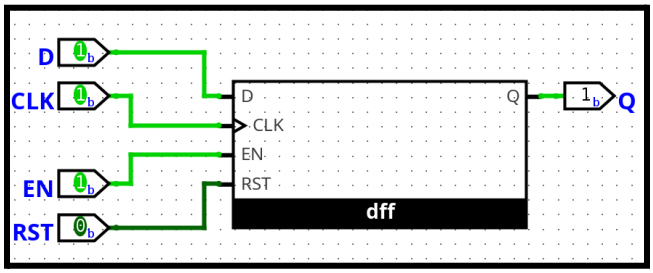
- ADDR[0], already split off from the word index for the fix above, is exactly the select line a TH6 byte-lane mux needs — 0 routes to the low byte, 1 to the high byte, with no extra inversion.
- It matches the byte order LIL/LIH already build a 16-bit constant in (low byte first, then high byte), keeping one convention across the whole processor instead of switching between register construction and memory layout.
- It matches x86/ARM-LE, which is what most of the group already has intuition for — fewer off-by-one mistakes once STB/LDB are being debugged.

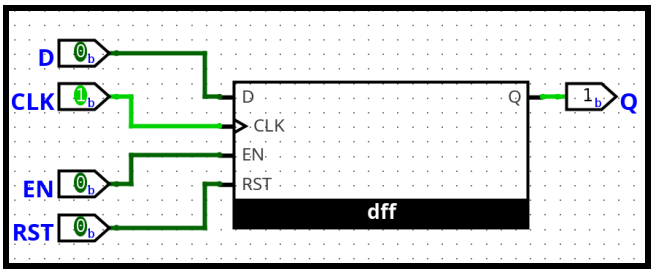
3. Test evidence

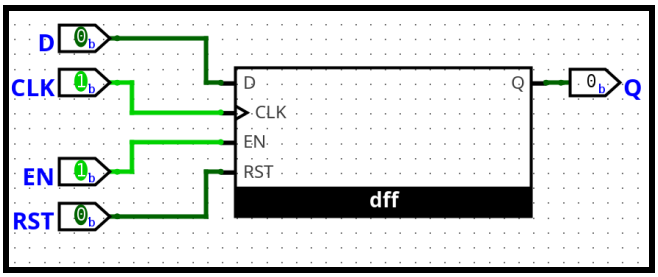
3.1 dff

#	Stimulus	Expected
1	RST=1	Q = 0
2	RST=0, D=1, EN=1, clock 0→1	Q = 1
3	D=0, EN=0, clock 0→1 (hold)	Q stays 1
4	D=0, EN=1, clock 0→1	Q = 0



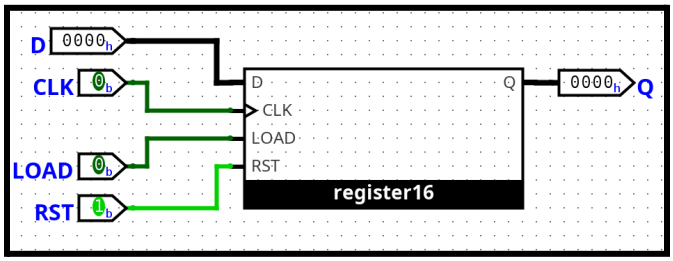


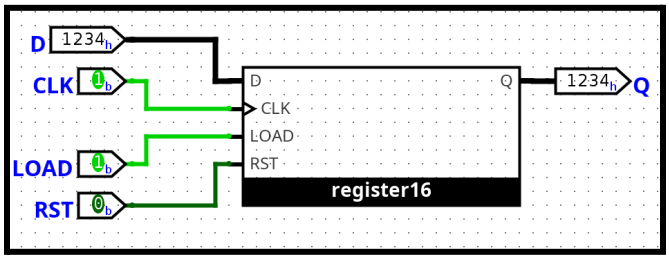


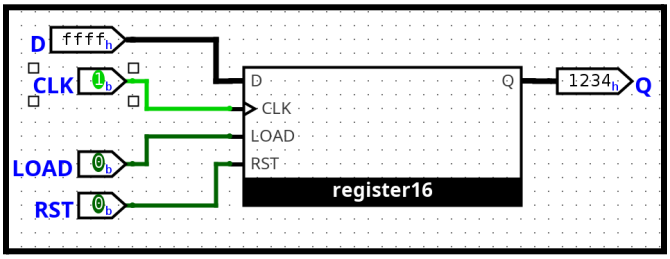


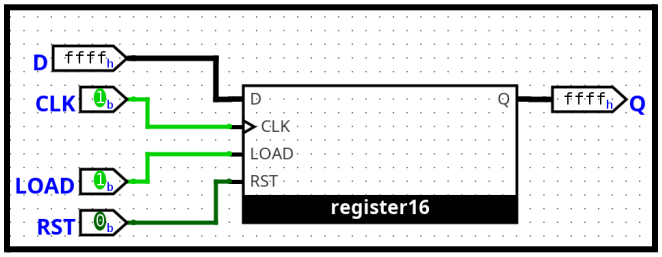
3.2 register16

#	Stimulus	Expected
1	RST=1	Q = 0x0000
2	LOAD=1, D=0x1234, clock 0→1	Q = 0x1234
3	LOAD=0, D=0xFFFF, clock 0→1 (hold)	Q stays 0x1234
4	LOAD=1, D=0xFFFF, clock 0→1	Q = 0xFFFF



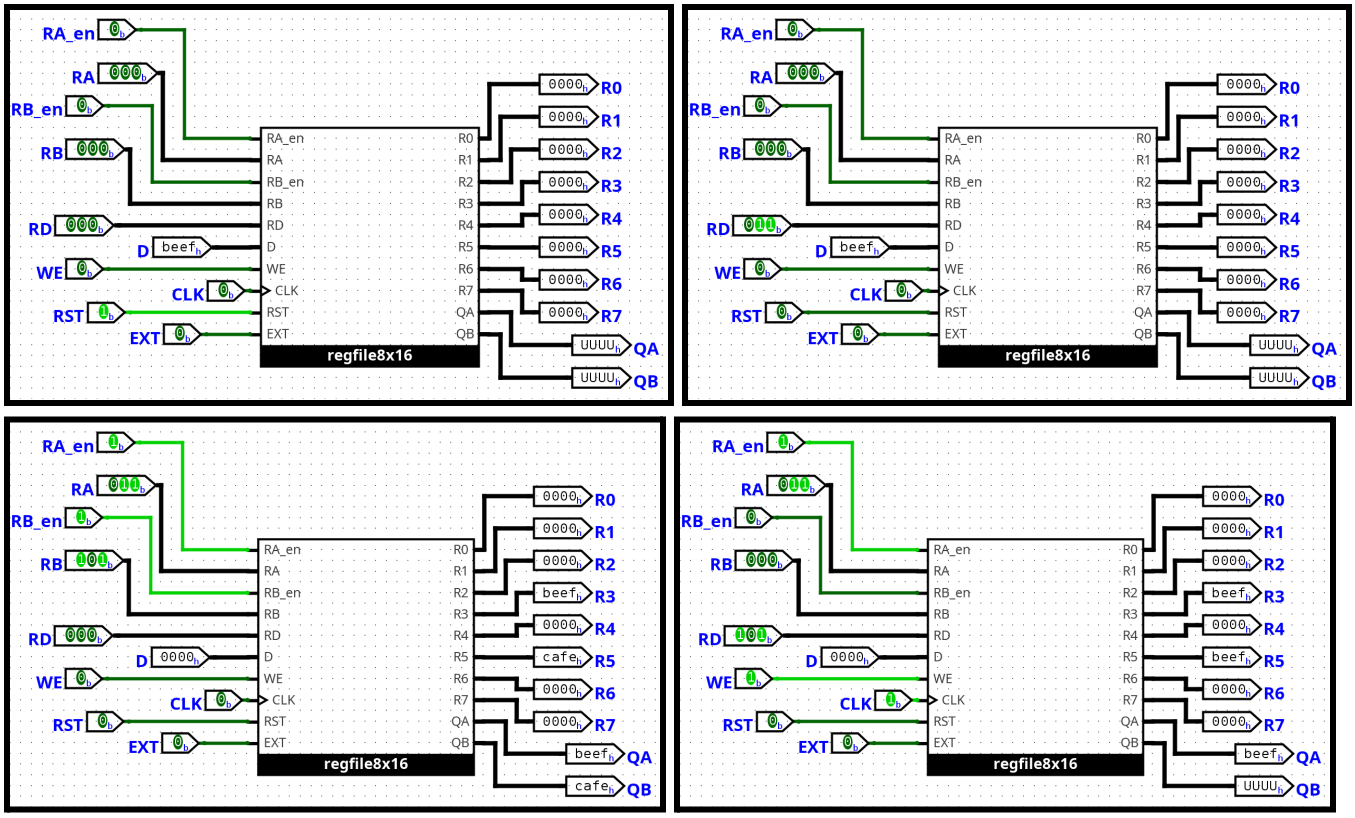






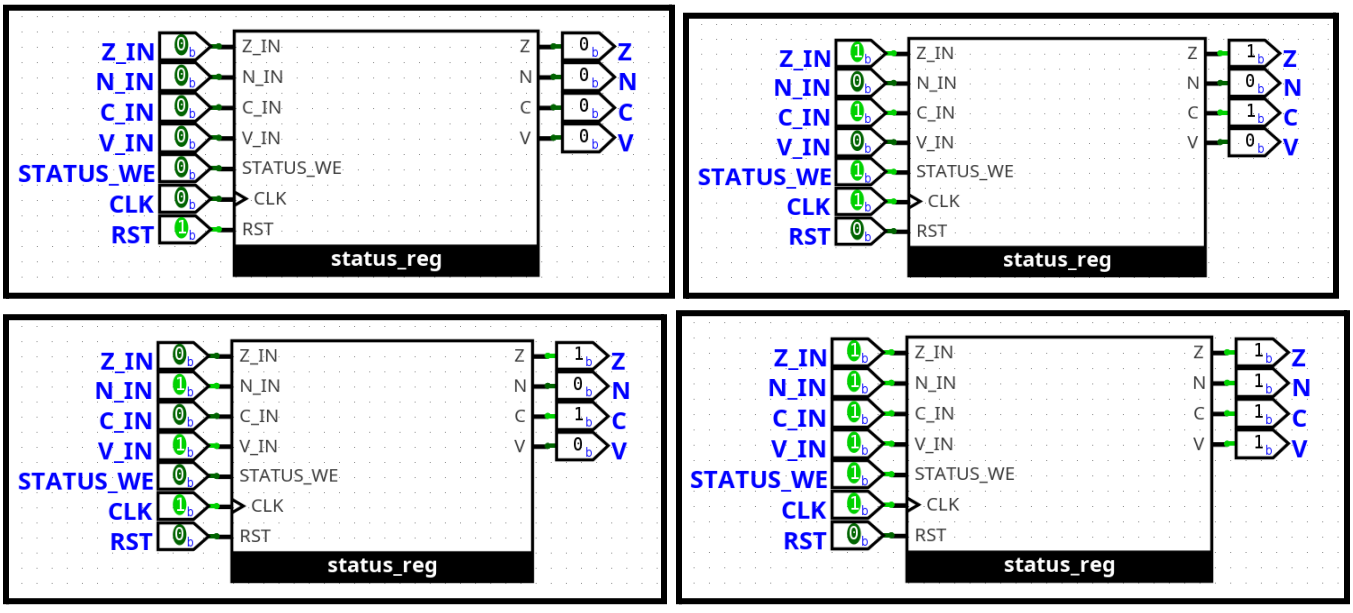
3.3 regfile8x16 — basic read/write and move (Here R0,R1…R7 outputs are shown in screenshots only in the test. Removed in final submission of circuit)

#	Stimulus	Expected
1	RST=1, then RA=RB=0..7 each enabled in turn	QA = QB = 0x0000 for every register
2	WE=1, RD=3, D=0xBEEF, clock 0→1, EXT=1	R3 now holds 0xBEEF
3	RA=3, RA_en=1, RB=5, RB_en=1 (R5 pre-loaded to 0xCAFE)	QA = 0xBEEF, QB = 0xCAFE simultaneously
4	RA=3, RA_en=1, RD=5, WE=1, EXT=0 (MOV R3, R5)(as per taracpu)	QA = 0xBEEF, R5=0xBEEF



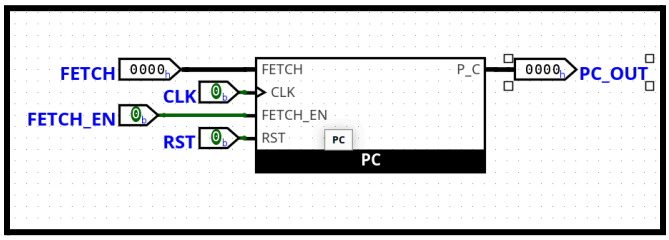
3.4 status_reg

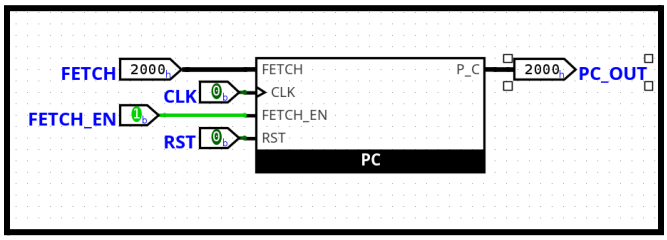
#	Stimulus	Expected
1	RST=1	Z=N=C=V=0
2	Z_IN=1,N_IN=0,C_IN=1,V_IN=0, STATUS_WE=1, clock 0→1	Z=1, N=0, C=1, V=0
3	All *_IN flipped, STATUS_WE=0, clock 0→1 (hold)	Outputs unchanged from step 2
4	All *_IN=1, STATUS_WE=1, clock 0→1	Z=N=C=V=1

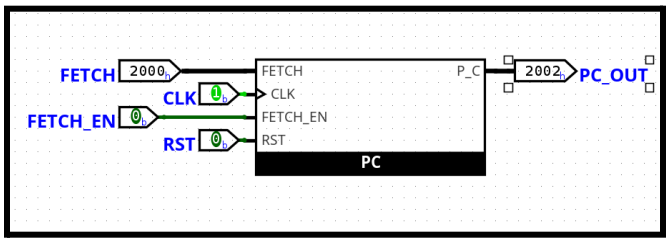


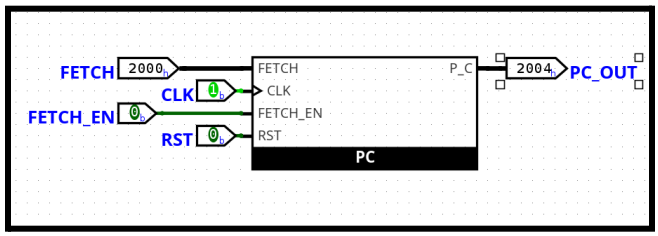
3.5 PC

#	Stimulus	Expected
1	RST=1	PC_OUT = 0x0000
2	FETCH_EN=1, FETCH=0x2000, clock 0→1	PC_OUT = 0x2000
3	FETCH_EN=0, FETCH=0x2000, clock 0→1	PC_OUT = 0x2002
4	FETCH_EN=0, FETCH=0x2000, clock 0→1	PC_OUT = 0x2004



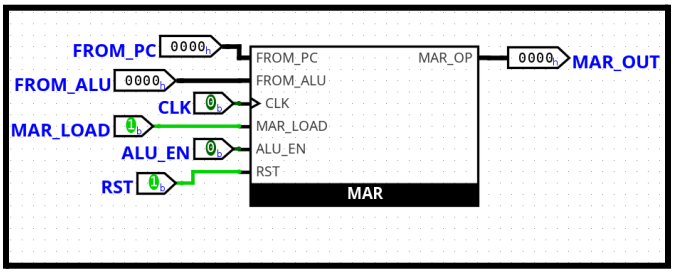


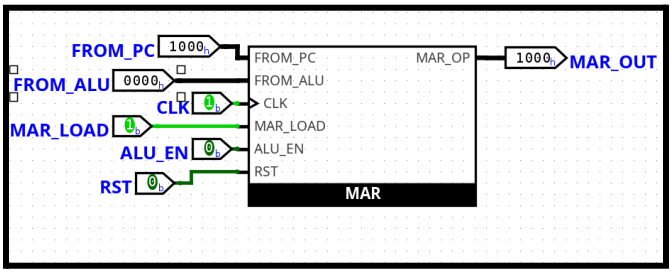


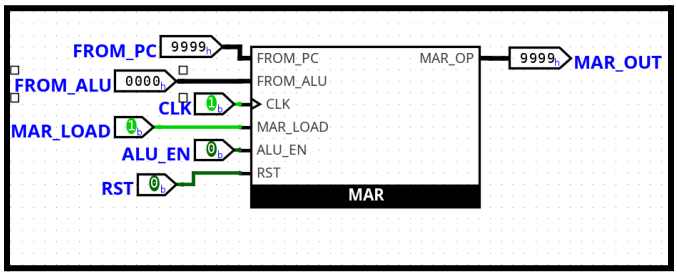


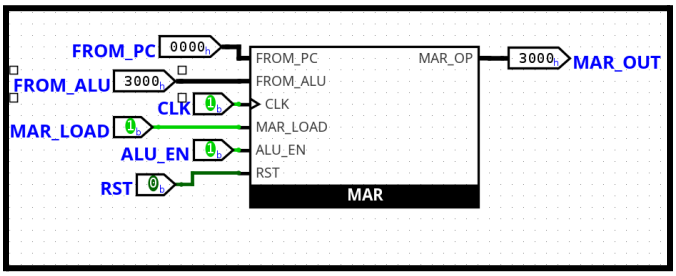
3.6 MAR

#	Stimulus	Expected
1	RST=1	MAR_OP = `0x0000`
2	RST=0, MAR_LOAD=1, ALU_EN=0, FROM_PC=`0x1000`, clock 0→1	MAR_OP = 0x1000
3	MAR_LOAD=0, FROM_PC=`0x9999`, clock 0→1	MAR_OP = `0x1000`
4	MAR_LOAD=1, ALU_EN=1, FROM_ALU=`0x3000`, clock 0→1	MAR_OP = `0x3000`



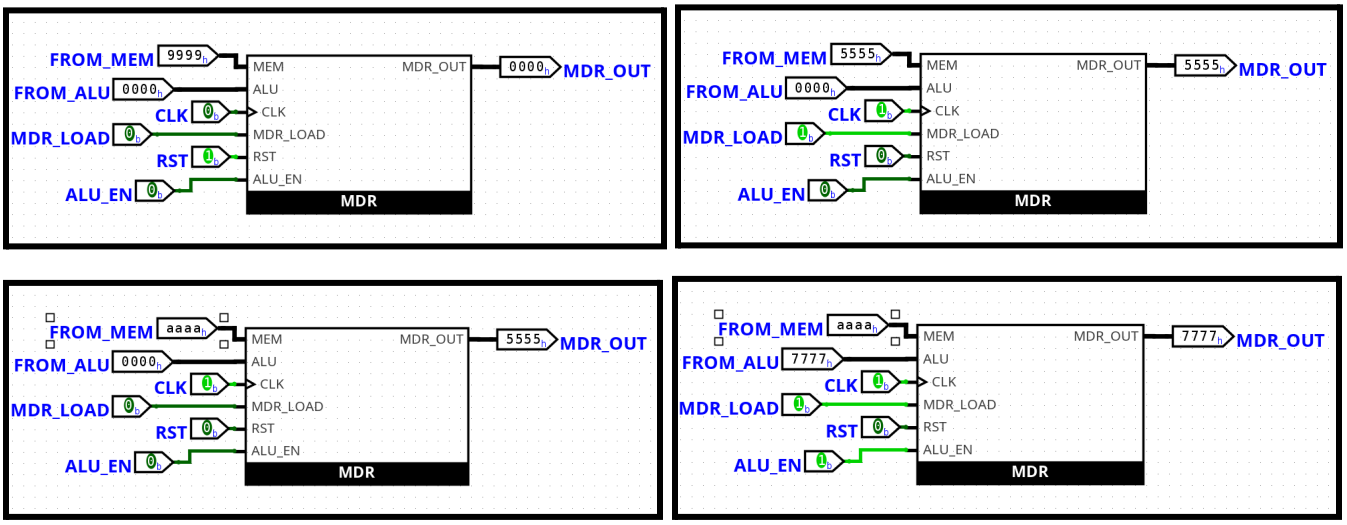






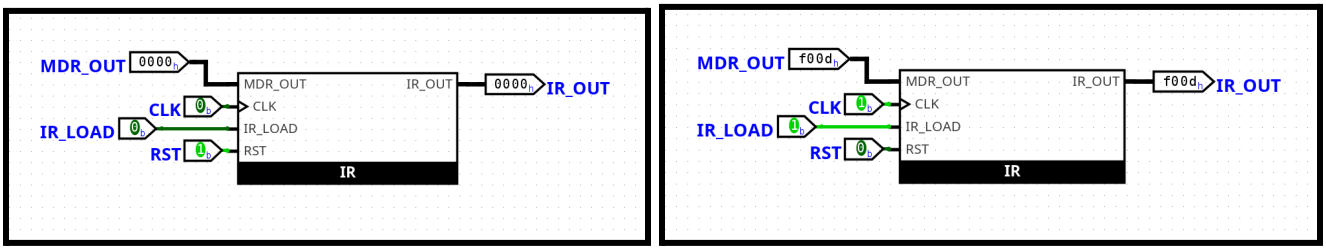
3.7 MDR

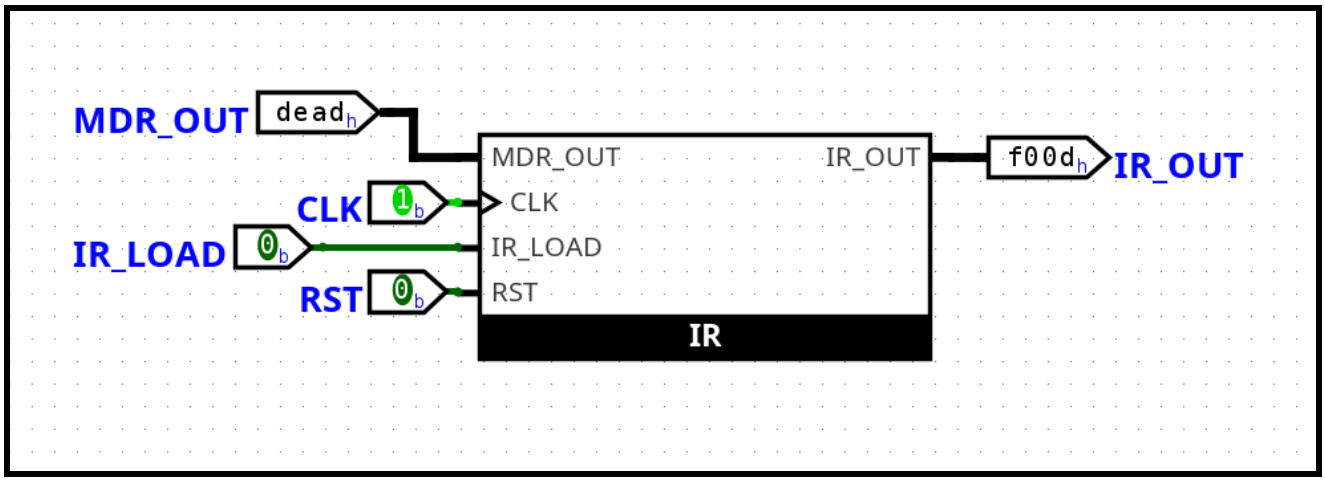
#	Stimulus	Expected
1	RST=1	MDR_OP = `0x0000`
2	RST=0, MDR_LOAD=1, ALU_EN=0, MEM=`0x5555`, clock 0→1	MDR_OP = 0x5555
3	MDR_LOAD=0, MEM=`0xAAAA`, clock 0→1	MDR_OP = `0x5555`
4	MDR_LOAD=1, ALU_EN=1, ALU=`0x7777`, clock 0→1	MDR_OP = `0x7777`



3.8 IR

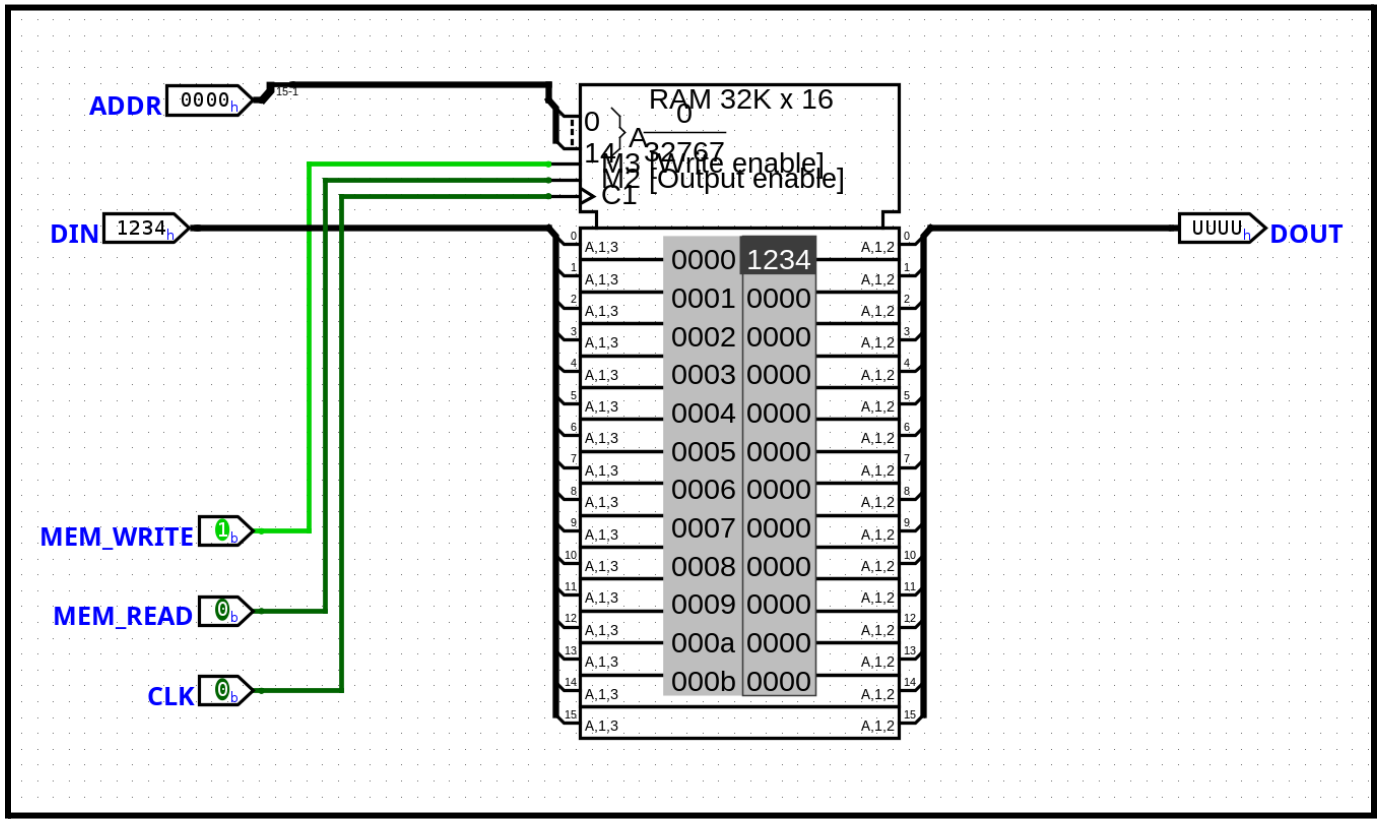
#	Stimulus	Expected
1	RST=1	IR_OP = `0x0000`
2	RST=0, IR_LOAD=1, MDR_OUT(input)=`0xF00D`, clock 0→1	IR_OP = 0xF00D
3	IR_LOAD=0, MDR_OUT(input)=`0xDEAD`, clock 0→1	IR_OP = `0xF00D`

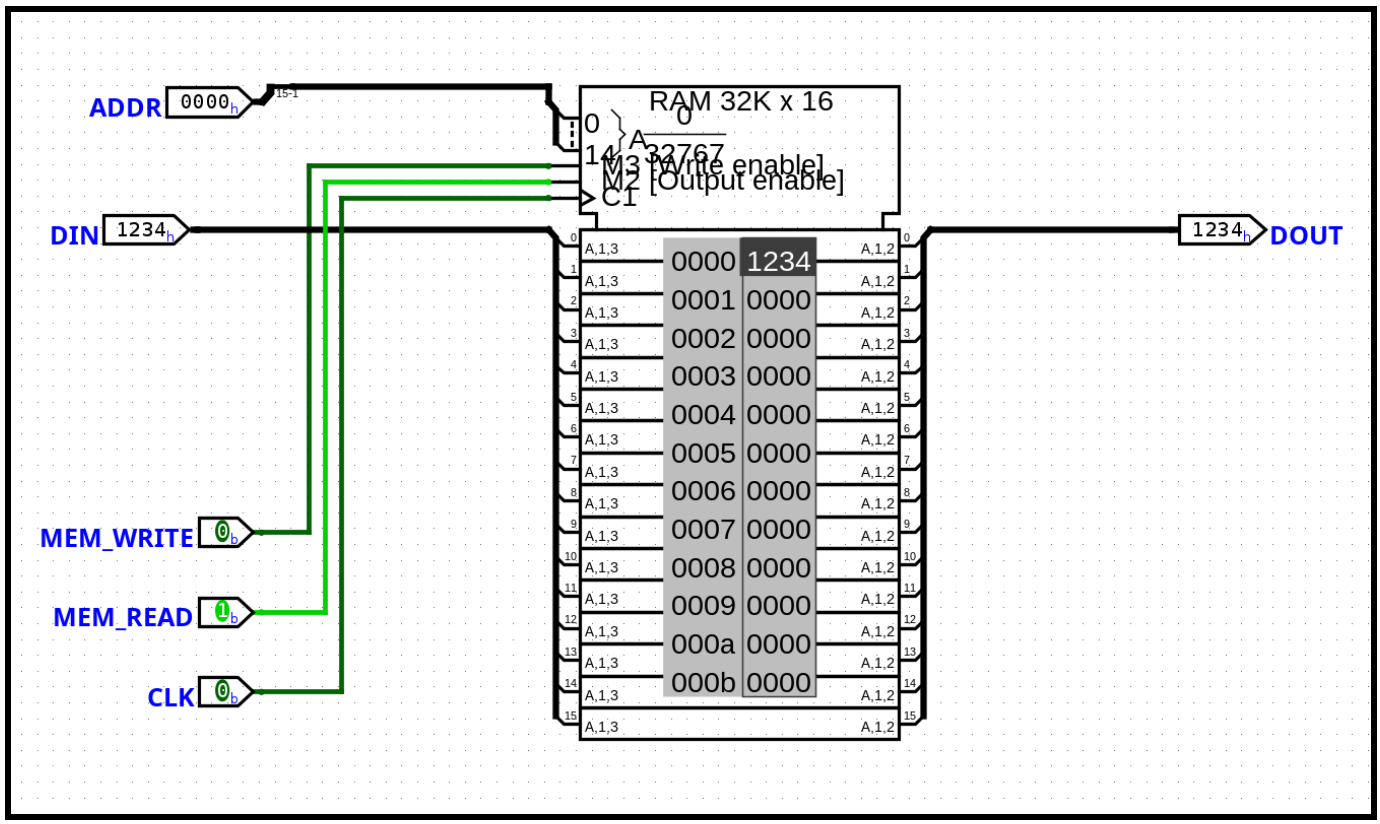


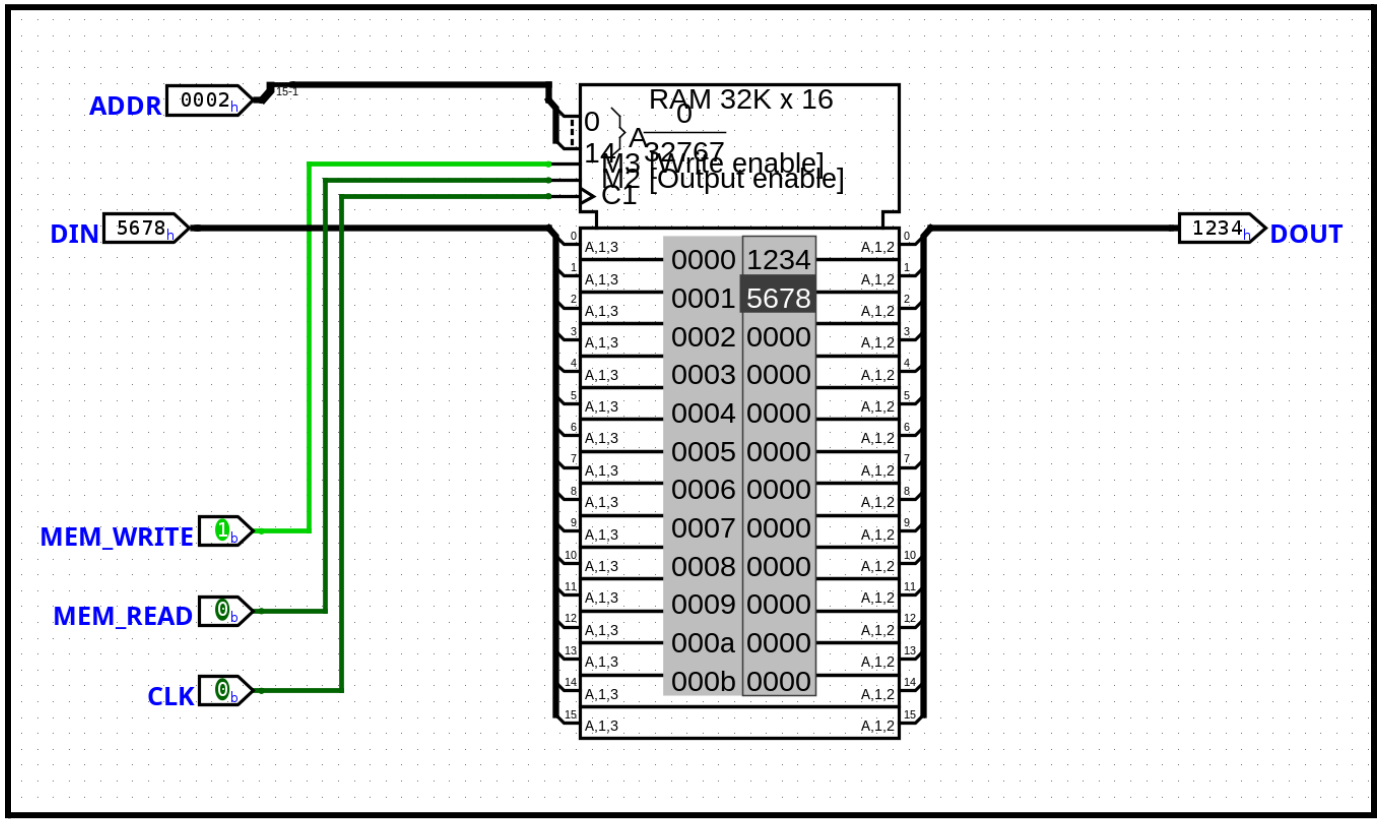


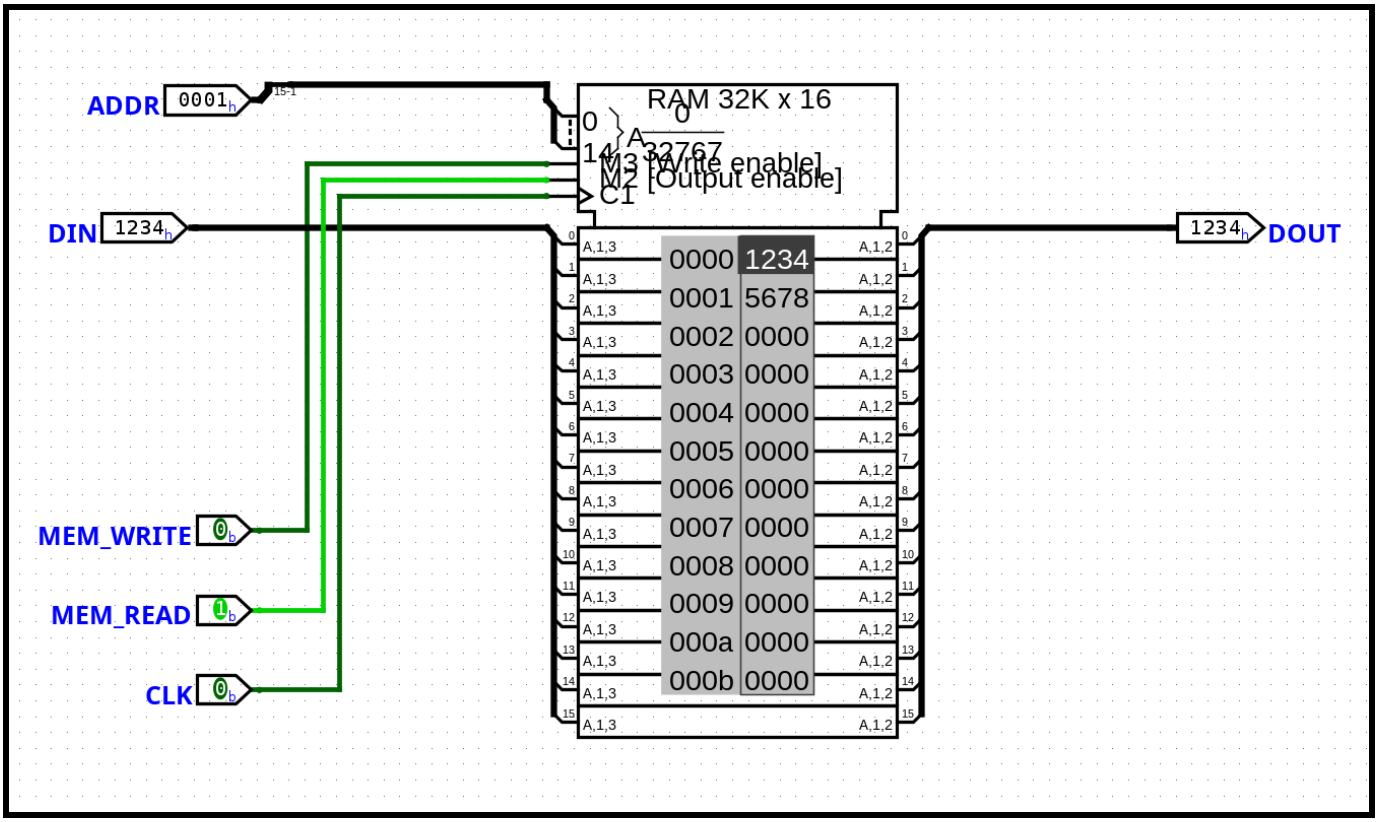
3.10 ram_word

#	Stimulus	Expected
1	MEM_WRITE=1, ADDR=0x0000, DIN=0x1234, clock 0→1	word stored at address 0
2	MEM_READ=1, ADDR=0x0000	DOUT = 0x1234
3	MEM_WRITE=1, ADDR=0x0002, DIN=0xABCD, clock 0→1; then read ADDR=0x0000	DOUT still = 0x1234 (address 2 didn't clobber address 0)
4	MEM_READ=1, ADDR=0x0001 (odd address, same word as step 1)	DOUT = 0x1234, identical to ADDR=0x0000

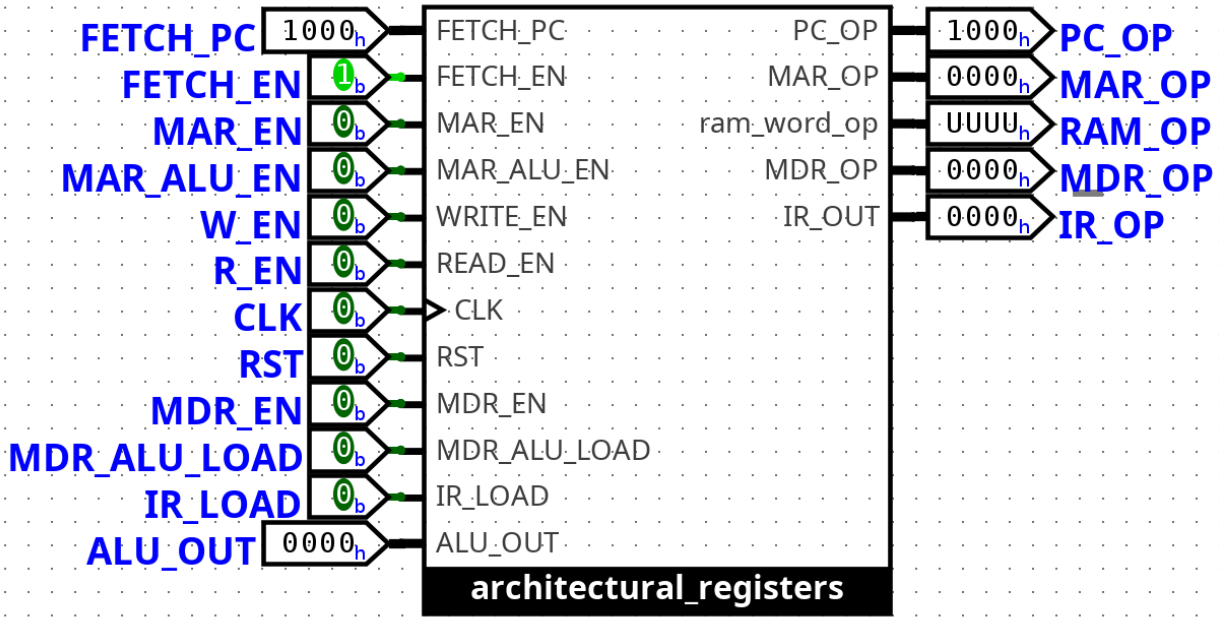


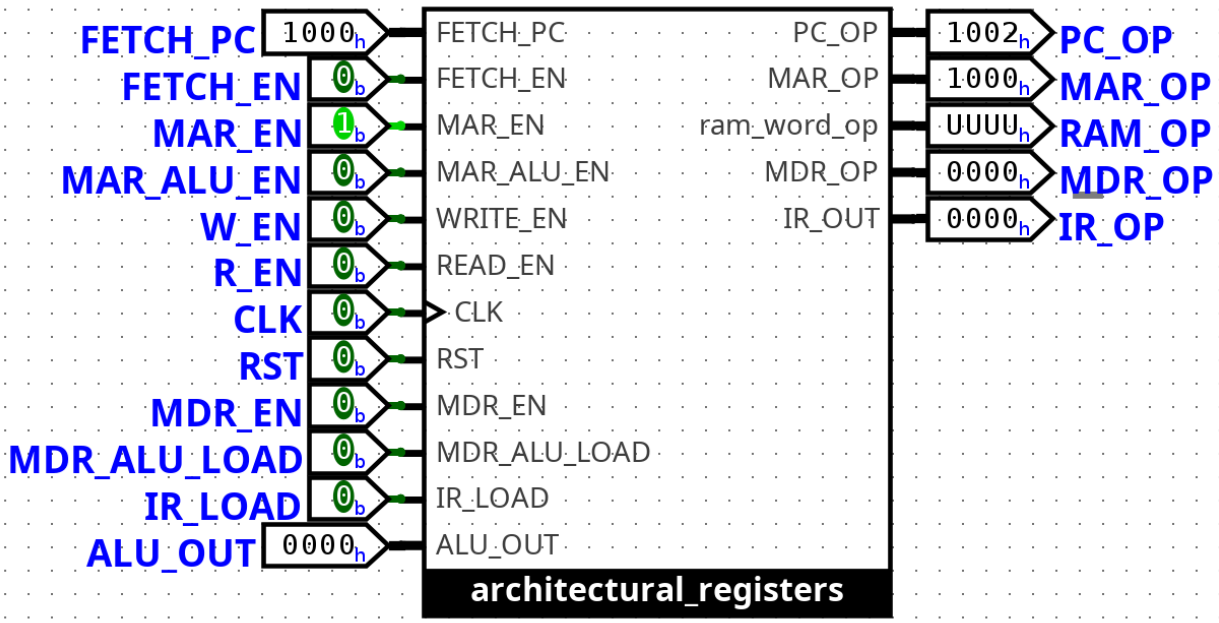

ADDR
0000h
15-1
DIN
1234h
MEM_WRITE
0b
MEM_READ
1b
CLK
0b
RAM 32K x 16
0
A
32767
M3 [Write enable]
M2 [Output enable]
CI
0000 1234
0001 0000
0002 0000
0003 0000
0004 0000
0005 0000
0006 0000
0007 0000
0008 0000
0009 0000
000a 0000
000b 0000
1234h
DOUT


ADDR
0002h
15-1
DIN
5678h
MEM_WRITE
1b
MEM_READ
0b
CLK
0b
RAM 32K x 16
0
A
32767
M3 [Write enable]
M2 [Output enable]
CI
0000 1234
0001 5678
0002 0000
0003 0000
0004 0000
0005 0000
0006 0000
0007 0000
0008 0000
0009 0000
000a 0000
000b 0000
1234h
DOUT



1	Seed PC	PC: FETCH_EN=1, FETCH=0x1000, clock 0→1	PC_OUT = 0x1000
2	T0 (MAR←PC)	MAR: ALU_EN=0, FROM_PC=0x1000, MAR_LOAD=1, clock 0→1	MAR_OP = 0x1000
3	T1 (MDR←Mem[MAR]; PC←PC+2)	ram_word: MEM_READ=1, ADDR=0x1000 (=MAR_OP); MDR: ALU_EN=0, MEM=DOUT, MDR_LOAD=1; PC: FETCH_EN=0, clock 0→1	MDR_OUT = 0x1234, PC_OUT = 0x1002
4	T2 (IR←MDR)	IR: MDR_OUT(input)=0x1234, IR_LOAD=1, clock 0→1	IR_OUT = 0x1234
5	T0′ (2nd fetch, MAR←PC)	MAR: FROM_PC=0x1002, MAR_LOAD=1, clock 0→1	MAR_OP = 0x1002
6	T1′	ram_word: ADDR=0x1002; MDR: MEM=DOUT, MDR_LOAD=1; PC: FETCH_EN=0, clock 0→1	MDR_OUT = 0x5678, PC_OUT = 0x1004
7	T2′ (IR←MDR)	IR: MDR_OUT(input)=0x5678, IR_LOAD=1, clock 0→1	IR_OUT = 0x5678


FETCH_PC
1000h
FETCH_EN
1b
MAR_EN
0b
MAR_ALU_EN
0b
W_EN
0b
R_EN
0b
CLK
0b
RST
0b
MDR_EN
0b
MDR_ALU_LOAD
0b
IR_LOAD
0b
ALU_OUT
0000h
FETCH_PC
FETCH_EN
MAR_EN
MAR_ALU_EN
WRITE_EN
READ_EN
CLK
RST
MDR_EN
MDR_ALU_LOAD
IR_LOAD
ALU_OUT
PC_OP
MAR_OP
ram_word_op
MDR_OP
IR_OUT
architectural_registers
1000h
PC_OP
0000h
MAR_OP
UUUUh
RAM_OP
0000h
MDR_OP
0000h
IR_OP


FETCH_PC
1000h
FETCH_EN
0b
MAR_EN
1b
MAR_ALU_EN
0b
W_EN
0b
R_EN
0b
CLK
0b
RST
0b
MDR_EN
0b
MDR_ALU_LOAD
0b
IR_LOAD
0b
ALU_OUT
0000h
FETCH_PC
FETCH_EN
MAR_EN
MAR_ALU_EN
WRITE_EN
READ_EN
CLK
RST
MDR_EN
MDR_ALU_LOAD
IR_LOAD
ALU_OUT
PC_OP
MAR_OP
ram_word_op
MDR_OP
IR_OUT
architectural_registers
1002h
PC_OP
1000h
MAR_OP
UUUUh
RAM_OP
0000h
MDR_OP
0000h
IR_OP

4. Faults encountered

Findings from inspecting the submitted .circ directly, in the format the rubric rewards — what broke, and how it was found.

Fault 1 — didn't wire read data in the register8x16 to D

Not rewiring data read to D pin meant that MOV couldn't be executed in a single clock cycle. Rewiring this fixed the issue.

Fault 2 — ram_word was not actually byte-addressed (resolved).

The spec calls for a byte-addressed wrapper presenting a 16-bit word at an even address. As originally built, ADDR was wired directly into a Logisim RAM primitive with addrWidth=16 — there was no shift or masking of the low address bit anywhere in the circuit, so ADDR=0x0000 and ADDR=0x0001 addressed two entirely independent words rather than the same one. So reading only Bit 1-15 fixed this issue (assuming Little Endian).